

From Human-in-the-Loop Testing to Code Summarization: Research Background and Interests

Amy Wei

Outline

- 3 Research Projects:
 - TerzoN: Human-in-the-Loop Software Testing with a Composite Oracle (FSE '25)
 - TestLoop: A Process Model Describing Human-in-the-Loop Software Test Suite Generation (TOSEM '25)
 - Sweat the Details: Exercise Breaks and AI Assistance in Code Summarization (under submission at TSE)

TerzoN: Human-in-the-Loop Software Testing with a Composite Oracle

(FSE '25)

Motivation

Fuzzing

Property-
based testing

Manual
testing

- **Testing** is difficult and time-consuming
- Different **testing tools have trade-offs**
 - **Fuzzing** cannot fully evaluate *correctness*
 - **Property-based testing** require users to *define properties to test*
 - **Manual testing** is labor-intensive and time-consuming
- Want to *easily* and *effectively* generate test suites while evaluating *correctness*
- We propose a **Composite Oracle**
 - Combines fuzzing (**implicit oracle**), property-based testing (**property-based oracle**), and manual testing (**example-based oracle**)
 - Returns a **single result** based on hierarchy of oracles

TerzoN

We implemented this Composite Oracle in a tool called **TerzoN**:

Fuzzing

Property-based testing

Manual testing

TS 2.ts

examples > 2 > TS 2.ts > ...

```
8  /**
9   * Converts a Roman numeral to a positive finite integer.
10  * Roman numeral is invalid, throws an exception.
11  *
12  * Roman numerals:
13  * - 'M':1000, 'D':500, 'C':100, 'L':50, 'X':10, 'V':5, 'I':1
14  * - When a "smaller" numeral appears anywhere before a "larger"
15  *   numeral, we subtract it (ex: 'IV' = -1+5 = 4).
16  *
17  * @param str Roman numeral
18  * @returns finite positive integer
19  */
20  export function decodeRoman(str: string): number {
21    var result = 0;
22    for (var i=0; i<weightsSymbols.length; ++i) {
23      var w = weightsSymbols[i][0];
24      var s = weightsSymbols[i][1];
25      var regex = RegExp(s);
26      while (str.match(regex)) {
27        result += w;
28        str = str.replace(regex, "");
29      }
30    }
31    return result;
32  }
```

Test: decodeRoman()

str: string =

Min length: 0, Max length: 3, Character set: MDCLXVI

Categorize output using:

☒ Heuristic validator, ☒ Property validator(s)

Test, More options...

Disagree 1, Failed 303, Passed 95

The property or human fuzzed these test cases

input: str	output	Heuristic	Property	Manual
'XCX'	120	✓	✗	✗
'VL'	55	✓	✗	✗
'VLX'	65	✓	✗	✗
expected output: 55				
'VC'	105	✓	✗	✗
expected output: 95				
'IL'	51	✓	✗	✗
expected output: 49				

1: TerzoN...
2: Test: decodeRoman()
3: str: string =
4: Categorize output using:
5: Test
6: More options...
7: The property or human fuzzed these test cases
8: Property-based, Manual

Implementation: TerzoN

- Challenge: how to **design** an **intuitive user interface**?
- Paper prototyping, piloting
 - How should users **select the oracles** they want to use?
 - How to display **passing / failing** tests?
 - What if two **oracles disagree**?
 - Results from **multiple validator functions**?

The screenshot shows the TerzoN test runner interface for a test named "Test: decodeRoman()". The interface includes input fields for "str: string =", "Min length" (0), "Max length" (3), and "Character set" (MDCLXVI). Below these are checkboxes for "Heuristic validator" and "Property validator(s)". A section titled "Oracles disagree" shows "Disagree 1", "Failed 303", and "Passed 95". A table displays test results for various inputs, with a blue box highlighting the "Multiple validators" column. Annotations 2 through 8 point to specific UI elements: 2 points to the test title, 3 points to the input fields, 4 points to the validator checkboxes, 5 points to the "Oracles disagree" section, 6 points to the "Disagree 1" count, 7 points to the "Failed 303" count, and 8 points to the "Multiple validators" column.

Test: decodeRoman()

str: string =

Min length: 0 Max length: 3 Character set: MDCLXVI

Categorize out: "larger"

Heuristic validator Property validator(s)

Oracles disagree

Disagree 1 Failed 303 Passed 95

Multiple validators

input: str	output	validators
'XCX'	120	✓ ⊗
'VL'	55	✓ ⊗
'VLX'	65	✓ ⊗
expected output: 55		
'VC'	105	✓ ⊗
expected output: 95		
'IL'	51	✓ ⊗
expected output: 49		

Evaluation and Results

- We *evaluated* TerzoN in a **human study** with 14 software engineers
- Participants used testing tools (**TerzoN** or **fast-check**) to generate test cases
- Selected 4 **TypeScript programs** with **bugs** from Rosetta Code, RunJS
 - Pilot studies: timing, prior knowledge, alignment with tools

Compared to fast-check, **participants using**

Participant comments:

- “[TerzoN] was much easier to use.”

- elicited **73% more bugs** ($p < 0.01$)

- I was responsible for:

- **Implementation**, interface, **experimental design**, **selecting** buggy programs, training materials, **pilot** studies, parts of **paper-writing**

Sweat the Details: Exercise Breaks and AI Assistance in Code Summarization

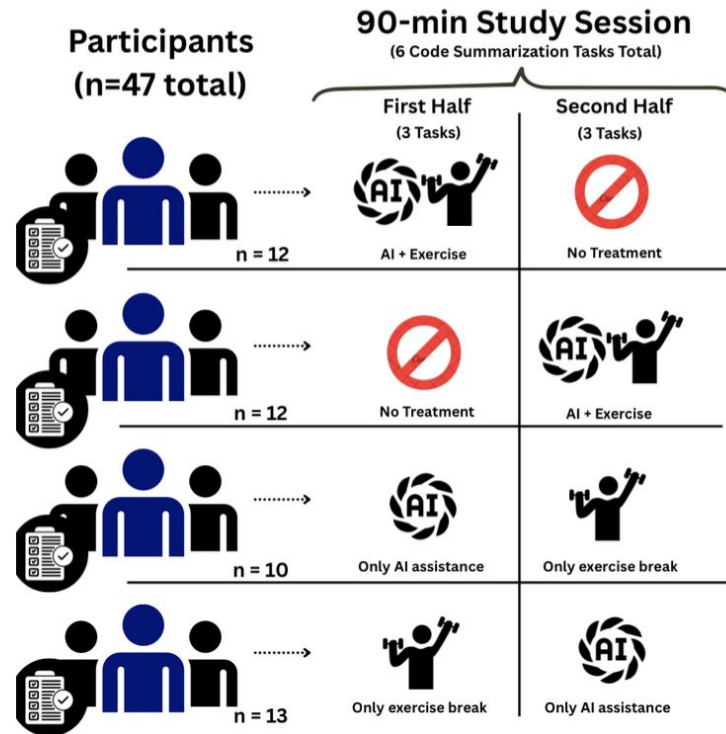
(under submission at TSE)

Motivation

- **Code summarization** is important in code maintenance and comprehension
- Real software contains bugs, so developers need to **identify defects** in code
- We want to understand the effects of **interventions** on developer **performance** for **code summarization** and **defect detection**
 - **Physical exercise** has been shown to **improve performance** in various fields, including SE
 - Advancements in **LLMs** are capable of **automatically summarizing** code, potentially supporting developers

Experimental Design

- Conducted a **human study** of 47 participants
- Task: *write a **code summary**, identify whether the code **contains a defect***
- Experimental conditions: brief **physical exercise** and **AI assistance** (randomized order, 2x2 factorial design)



Experiment Stimuli

- Selected 17 Java modules from GitHub
- Seeded** the source code with **defects**
- Generated **AI responses** from ChatGPT-4o (summary of the code, description of whether it contains a defect)
 - 2 versions of each AI response:
correct and **incorrect** (to represent hallucinations; used prompt engineering)

```
413 ChatCompletionRequest createRequest(Prompt prompt, boolean stream) {
414
415     List<ChatCompletionMessage> chatCompletionMessages = prompt.getInstructions().stream().map(message -> {
416         if (message.getMessageType() == MessageType.USER || message.getMessageType() == MessageType.SYSTEM) {
417             Object content = message.getContent();
418             if (message instanceof UserMessage userMessage) {
419                 if (!CollectionUtils.isEmpty(userMessage.getMedia())) {
420                     content = convertMediaContent(userMessage);
421                 }
422             }
423
424             return List.of(new ChatCompletionMessage(content,
425                 ChatCompletionMessage.Role.valueOf(message.getMessageType().name()));)
426         }
427         else if (message.getMessageType() == MessageType.ASSISTANT) {
428             var assistantMessage = (AssistantMessage) message;
429             List<ToolCall> toolCalls = null;
430             if (!CollectionUtils.isEmpty(assistantMessage.getToolCalls())) {
431                 toolCalls = assistantMessage.getToolCalls().stream().map(toolCall -> {
432                     var function = new ChatCompletionFunction(toolCall.name(), toolCall.arguments());
433                     return new ToolCall(toolCall.id(), toolCall.type(), function);
434                 }).toList();
435             }
436             return List.of(new ChatCompletionMessage(assistantMessage.getText(),
437                 ChatCompletionMessage.Role.ASSISTANT, null, null, toolCalls, null));
438         }
439     });
440 }
```

1. Write a code summary for the method `createRequest()`. [Click here to view the source code.](#)

The summary will be assessed according to the following quality attributes:

- Summary accuracy
- Whether the summary is missing information
- Whether the summary contains unnecessary information
- Summary readability

Please write your summary in the box below:

2. Identify whether any method in the code shown contains a bug.

Here, a bug is a disconnect between what the developer intended for the method to do and what it actually does. If we were to execute code that contains a bug, the system would do something it's not meant to do.

- ☐ Yes
☐ No

Our AI assistant generated the following text about this method:

(1) **Summary of createRequest Method:**

The createRequest method constructs a ChatCompletionRequest from the given Prompt. It processes the prompts messages and converts them into a list of ChatCompletionMessage objects. Depending on the message type, it handles user, system, assistant, and tool messages differently, including media content conversion and tool call handling. The method also merges runtime options with default options to ensure the request parameters are accurately configured for the chat API call. Support for streaming and various configurations like model, temperature, and token settings are included.

(2) **Bug Identification:**

yes
Incorrect variable usage

The createRequest method references requestOptions without initializing it before using it to resolve tool definitions and set request parameters. The variable requestOptions should be initialized with merged options before it is used in the method.

Results

- Analyzed using GLMER (generalized linear mixed-effects regression)
- **Physical exercise impairs** code generation
 - Longer response times ($p < 0.001$)
 - Reduced summary accuracy ($p < 0.001$)
- Exercise effects **diminish** over repeated trials ($p < 0.04$)
- **AI assistance improves** conciseness ($p = 0.002$) and completeness of summaries ($p = 0.039$)

Possible explanation: Exercise causes an **increased heart rate**, which may interfere with **concentration** immediately after.

Participant self-reporting: For the prompt “The exercise break **helped me concentrate** on this task,” 47% disagreed (60/128), 35% agreed (45/128), 18% neutral (23/128).

- I was responsible for:
 - Generating **problem**, **literature** review, **experimental design**, study **stimuli** (defect-seeded source code, AI responses), **pilot** studies, **recruiting**, **conducting** study, parts of **analysis**, **paper-writing**